Table des matières

Exercices corrigés 10 — Index, plans d'exécution et optimisation 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 5
Mises en situation 5
Corrigé 7

Exercices corrigés 10 — Index, plans d'exécution et optimisation

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL et lire des plans ; les diagnostics vous montrent du code ou des plans qui clochent ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez trois heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Ce module n'a qu'un laboratoire, sur PostgreSQL. Un énoncé marqué `[OLTP]` se traite donc dans PostgreSQL sur `Adventureworks`, ou sur le schéma `labo10` que vous avez construit. Les rares énoncés marqués `[OLAP]` se traitent dans DuckDB, sur l'entrepôt. Sans marque, la question ne demande pas de moteur.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Pourquoi mesure-t-on deux ou trois fois avant de conclure qu'une requête est plus rapide ?
2. Un index B-tree est une structure triée. Citez trois choses que ce tri lui permet de faire.
3. Nommez les trois coûts d'un index, et dites lequel est payé même quand personne ne lit la table.
4. `EXPLAIN` et `EXPLAIN ANALYZE`. Quelle est la différence, et quelle précaution prendre avec le second sur un `UPDATE` ?
5. Dans quel sens se lit un plan d'exécution ?

© 2026 iAlexMG. Tous droits réservés.

6. Quel est le signal de diagnostic le plus utile d'un plan, et que veut dire un écart d'un facteur cent ?
7. Sur une boucle imbriquée, pourquoi ne faut-il pas lire le temps du nœud interne tel quel ?
8. Donnez trois nœuds de jointure et dites dans quel cas chacun est le bon choix.
9. Définissez la sargabilité en une phrase, sans employer le mot anglais dont elle vient.
10. Réécrivez `WHERE EXTRACT(YEAR FROM orderdate) = 2023` de façon sargable, et dites ce que la réécriture a déplacé.
11. `WHERE nom LIKE '%150'` ne peut employer aucun index B-tree. Pourquoi ?
12. Un index B-tree ordinaire sur du texte ne sert pas toujours `LIKE 'préfixe%'`. Donnez la cause et la parade.
13. Un développeur corrige une erreur de type en écrivant `WHERE cle::int = 29825`. Qu'a-t-il cassé, et qu'aurait-il fallu écrire ?
14. Énoncez la règle du préfixe le plus à gauche, avec une analogie, puis dites ce que PostgreSQL 18 fait d'un filtre qui saute la première colonne de l'index.
15. Dans un index composite, où placez-vous la colonne filtrée par égalité et où placez-vous la plage ? Justifiez.
16. Qu'est-ce qu'un index couvrant, et à quoi sert le mot-clé `INCLUDE` ?
17. Citez les quatre situations où un index nuit plus qu'il n'aide, et donnez la parade pour celle de la colonne peu sélective.
18. Comment sait-on qu'un index de production ne sert jamais ?

1. Sur quoi le planificateur s'appuie-t-il pour choisir un plan, et dans quelles trois situations cette information le trompe-t-elle ?
2. L'entrepôt n'a pas d'index utile. Expliquez ce qui remplace l'index, dites ce qu'il faut regarder pour savoir si cela a fonctionné, et pourquoi ce diagnostic est plus difficile que sur le transactionnel.

Questions pratiques

P1. `[OLTP]` Sur le schéma `labo10`, écrivez la requête qui trouve les dix commandes du client 29825 en 2023, puis mesurez-la sans index. Créez ensuite l'index qui la sert, remesurez, et donnez le rapport entre les deux temps.

P2. `[OLTP]` Écrivez trois requêtes qui renvoient exactement le même résultat, dont une seule emploie un index existant. Prouvez-le par les plans.

P3. `[OLTP]` Concevez l'index le mieux adapté à cette requête, et justifiez l'ordre des colonnes :

```
SELECT ligne_id, linetotal
FROM   labo10.grosse
```


© 2026 iAlexMG. Tous droits réservés.

```sql
WHERE  customerid = 29825 AND jour >= '2023-01-01'
ORDER  BY jour;
```

P4. [OLTP] Faites de l'index de P3 un index couvrant. Vérifiez que le plan devient un parcours d'index seul, et dites ce qui se passerait si vous ajoutiez `produit` au `SELECT`.

P5. [OLTP] Écrivez la requête qui liste, pour le schéma `labo10`, chaque index avec sa taille et son nombre d'utilisations, du moins utilisé au plus utilisé.

P6. [OLTP] Créez un index partiel qui ne couvre que les lignes de plus de 1 000 $, puis écrivez une requête qu'il sert et une requête très proche qu'il ne sert pas. Expliquez la frontière. (avancé)

P7. [OLAP] Sur l'entrepôt, écrivez deux requêtes équivalentes sur une table rangée par date, dont une seule permet l'élagage. Mesurez les deux et dites ce que vous regardez pour le prouver.

P8. [OLTP] Une table de 40 millions de lignes vient d'être chargée. Écrivez la séquence de commandes que vous lancez avant de mesurer quoi que ce soit, et justifiez chaque commande en une phrase. (avancé)

Diagnostics

E1. Sur le schéma `labo10`, avec l'index `idx_grosse_produit` de la partie 2, cette requête joint les lignes de ventes à leurs en-têtes de commande :

```sql
SELECT SUM(h.freight)
FROM   labo10.grosse AS g
JOIN   sales.salesorderheader AS h ON h.salesorderid = g.salesorderid
WHERE  g.productid = 870 AND g.produit = 'Water Bottle - 30 oz.' AND g.unitprice = 4.99;
```

PostgreSQL 18 rend ce plan, lancé par `EXPLAIN ANALYZE` :

```
Aggregate  (cost=16058.26..16058.27 rows=1 width=32) (actual time=77.817..77.817 rows=1.00 loops=1)
  Buffers: shared hit=131162 read=65
  ->  Nested Loop  (cost=529.66..16057.91 rows=139 width=6) (actual time=1.589..72.688 rows=42440.00 loops=1)
        Buffers: shared hit=131162 read=65
        ->  Bitmap Heap Scan on grosse g  (cost=529.38..15195.51 rows=139 width=4) (actual time=1.566..15.618 rows=42440.00
loops=1)
              Recheck Cond: (productid = 870)
              Filter: (((produit)::text = 'Water Bottle - 30 oz.'::text) AND (unitprice = 4.99))
              Rows Removed by Filter: 4440
              Heap Blocks: exact=3865
              Buffers: shared hit=3907
              ->  Bitmap Index Scan on idx_grosse_produit  (cost=0.00..529.34 rows=48122 width=0) (actual time=1.136..1.136
rows=46880.00 loops=1)
```


© 2026 iAlexMG. Tous droits réservés.

```
                  Index Cond: (productid = 870)
                  Index Searches: 1
                  Buffers: shared hit=42
        ->  Index Scan using "PK_SalesOrderHeader_SalesOrderID" on salesorderheader h  (cost=0.29..6.20 rows=1 width=10) (actual
time=0.001..0.001 rows=1.00 loops=42440)
              Index Cond: (salesorderid = g.salesorderid)
              Index Searches: 42440
              Buffers: shared hit=127255 read=65
Execution Time: 77.908 ms
```

Le moteur a choisi une boucle imbriquée, et la requête met plus de deux fois le temps qu'elle prend en `Hash Join`, en touchant vingt-huit fois plus de pages. Expliquez ce qui l'a conduit à ce choix, et dites ce que vous feriez.

E2. Un développeur signale que son index ne sert pas :

```sql
CREATE INDEX idx_jour ON commande (jour);
SELECT COUNT(*) FROM commande WHERE jour + INTERVAL '1 day' > now();
```

Expliquez et corrigez.

E3. Sur `labo10.grosse`, un index a été créé ainsi, pour servir deux requêtes, et c'est le seul index de la table :

```sql
CREATE INDEX idx_jour_client ON labo10.grosse (jour, customerid);
-- Requête A
SELECT COUNT(*) FROM labo10.grosse WHERE customerid = 29825;
-- Requête B
SELECT COUNT(*) FROM labo10.grosse WHERE jour >= '2023-01-01' AND customerid = 29825;
```

Sous PostgreSQL 18, les deux plans emploient l'index, et son auteur en conclut qu'il sert bien les deux requêtes. Les plans affichent pourtant `Index Searches: 414` pour A et `Index Searches: 398` pour B. Dites ce que ces nombres comptent, pourquoi ils sont si élevés, si l'auteur a raison, et quel index vous créeriez à la place.

E4. Sur le schéma `labo10`, un index sur `grosse (salesorderid)` sert cette jointure des commandes de juin 2025 à leurs lignes :

```sql
CREATE INDEX idx_grosse_commande ON labo10.grosse (salesorderid);

SELECT h.salesorderid, g.linetotal
FROM   sales.salesorderheader AS h
JOIN   labo10.grosse AS g ON g.salesorderid = h.salesorderid
WHERE  h.orderdate >= '2025-06-01';
```

Son plan, par `EXPLAIN ANALYZE` :

© 2026 iAlexMG. Tous droits réservés.

```
Nested Loop  (cost=0.43..9328.67 rows=35896 width=11) (actual time=2.065..7.596 rows=20470.00 loops=1)
  Buffers: shared hit=4558
  ->  Seq Scan on salesorderheader h  (cost=0.00..1115.31 rows=931 width=4) (actual time=2.056..2.164 rows=905.00 loops=1)
        Filter: (orderdate >= '2025-06-01 00:00:00'::timestamp without time zone)
        Rows Removed by Filter: 30560
        Buffers: shared hit=722
  ->  Index Scan using idx_grosse_commande on grosse g  (cost=0.43..8.20 rows=62 width=11) (actual time=0.001..0.003 rows=22.62
loops=905)
        Index Cond: (salesorderid = h.salesorderid)
        Index Searches: 905
        Buffers: shared hit=3836
Execution Time: 8.143 ms
```

Un collègue conclut que le parcours d'index ne pèse rien, puisqu'il prend 0,003 milliseconde, et qu'il faut chercher le temps ailleurs. Corrigez-le, chiffres à l'appui, puis dites si la boucle imbriquée était le bon choix.

E5. Une équipe a ajouté onze index sur une table d'événements alimentée par un flux temps réel. Les lectures sont un rapport quotidien. Les insertions ont ralenti d'un facteur quatre. Décrivez la démarche que vous suivriez, dans l'ordre, et dites quelle requête vous écririez en premier. (avancé)

E6. Sur l'entrepôt, une requête filtrée sur une seule journée met le même temps qu'un balayage complet. Le développeur en déduit qu'il faut réécrire toute la table dans un autre ordre. Donnez deux causes plus probables à vérifier avant, et dites comment vous les vérifiez.

Questions à développer

A1. La théorie compare un index à un abonnement plutôt qu'à un achat. Développez cette image en un paragraphe, puis dites quelles données vous rassembleriez pour décider de supprimer un index existant en production. Précisez ce que vous feriez avant la suppression pour pouvoir revenir en arrière.

A2. Un architecte propose une règle simple : « on indexe toutes les colonnes qui apparaissent dans un `WHERE` ». Répondez-lui en un paragraphe, en reconnaissant ce que la règle a de juste, puis proposez une règle de remplacement qui tienne en deux phrases.

A3. Comparez la façon dont chacun des deux terrains rend un filtre rapide. Votre réponse doit expliquer pourquoi l'un demande une structure explicite et l'autre non, dire ce que le second ne dispense pas de faire, et nommer la seule notion du module qui vaut identiquement des deux côtés.

A4. Le module affirme que l'écart entre les lignes estimées et les lignes réelles est le meilleur signal de diagnostic. Expliquez pourquoi cet écart est plus informatif que le temps d'exécution, et décrivez trois causes possibles d'un écart d'un facteur cent, avec le traitement de chacune. (avancé)

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

M1. Fonderie Bécancour, le rapport qui s’est mis à ramer. Le rapport de production tourne chaque nuit depuis quatre ans. Il lisait une table de coulées en huit minutes ; depuis six semaines, il en met cinquante, et parfois il n’a pas fini quand l’équipe du matin arrive. Personne n’a modifié le SQL. La table a grossi de trente pour cent sur la période, et une nouvelle application écrit dedans en continu depuis deux mois.

Décrivez votre démarche de diagnostic, dans l’ordre, en nommant à chaque étape la commande ou la vue système que vous consultez. Formulez au moins trois hypothèses compatibles avec les faits de l’énoncé, et dites pour chacune la mesure qui la confirme ou l’écarte. Précisez ce que vous ne feriez pas en premier, et pourquoi.

M2. Grossiste alimentaire Yamaska, l’index qu’on vous demande. Un développeur vous écrit que la page de recherche des commandes est lente, et vous demande d’ajouter un index sur `date_livraison`. La page permet de filtrer par entrepôt, par statut, par plage de dates de livraison et par nom de client partiel, et l’utilisateur remplit rarement plus de deux de ces quatre champs. La table compte 90 millions de lignes et reçoit 400 écritures par seconde.

Répondez au développeur. Votre réponse doit dire ce que vous demandez avant d’agir, expliquer pourquoi un seul index sur `date_livraison` a peu de chances de régler le problème, et proposer une stratégie d’indexation qui tienne compte des quatre champs sans en créer quatre. Traitez explicitement le cas du nom de client partiel, qui est le plus délicat. Dites enfin ce que vous surveilleriez après la mise en place.

M3. Billetterie Matapédia, la migration vers l’entrepôt. Matapédia déplace ses rapports d’une base PostgreSQL vers un entrepôt. L’équipe a passé trois ans à ajuster une trentaine d’index sur la base transactionnelle, et le responsable technique demande la liste des index à recréer dans l’entrepôt pour ne pas perdre les gains.

Répondez-lui. Expliquez ce qui remplace les index, ce qui se transpose et ce qui ne se transpose pas, et ce que devient l’expertise accumulée par l’équipe : dites précisément quelle partie de son travail reste valable et quelle partie devient sans objet. Proposez ensuite le petit nombre de décisions qu’il faut réellement prendre sur l’entrepôt, et dites comment vous les valideriez par la mesure plutôt que par l’intuition. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Parce que la première exécution paie le chargement des pages en mémoire. Une requête qui passe de 900 à 40 millisecondes n'a peut-être rien gagné d'autre qu'un cache chaud. On compare des exécutions comparables, et on retient la meilleure de chaque côté.

Théorie : `Theorie_10_SQL.md`, section 1.

2. Trouver une valeur exacte en quelques comparaisons, trouver une plage de valeurs puisque les valeurs voisines sont voisines dans l'index, et rendre les lignes déjà triées, ce qui rend un `ORDER BY` sur la colonne gratuit. Une quatrième, si l'index contient toutes les colonnes demandées : répondre sans ouvrir la table.

Théorie : `Theorie_10_SQL.md`, section 2.

3. L'espace disque, le ralentissement de chaque écriture, et la place occupée en mémoire au détriment des autres données. Le deuxième est payé même si personne ne lit jamais la table, ce qui est exactement ce qui rend un index inutile coûteux.

Théorie : `Theorie_10_SQL.md`, section 6.

4. `EXPLAIN` montre le plan choisi et les coûts estimés sans rien exécuter. `EXPLAIN ANALYZE` exécute vraiment la requête et ajoute les temps et les comptes réels, et, depuis PostgreSQL 18, les pages lues par chaque nœud. Sur un `UPDATE`, cela veut dire que les lignes sont modifiées : il faut envelopper la commande dans une transaction qu'on annule.

Théorie : `Theorie_10_SQL.md`, section 3.

5. De l'intérieur vers l'extérieur : le nœud le plus indenté s'exécute en premier, et son résultat remonte au nœud parent.

Théorie : `Theorie_10_SQL.md`, section 3.

6. L'écart entre le nombre de lignes estimé et le nombre réel. Un facteur cent veut dire que le planificateur a choisi son plan sur une information fausse : le problème est du côté des statistiques, pas du côté des index, et ajouter un index ne réglera rien.

Théorie : `Theorie_10_SQL.md`, sections 3 et 7.

7. Parce que le temps affiché est celui d'une itération. Il faut le multiplier par le nombre de boucles, que le plan donne sous `loops`. Un nœud à 0,03 milliseconde répété 400 000 fois coûte douze secondes. Le compte de lignes du nœud est lui aussi une moyenne par boucle, que PostgreSQL 18 affiche avec deux décimales.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_10_SQL.md`, section 3.

8. La boucle imbriquée quand la table externe, celle que la boucle parcourt, est petite et que la table interne, celle où l'on cherche, est indexée. La jointure de hachage quand les deux tables sont grosses et qu'aucune n'est indexée utilement. La jointure par fusion quand les deux côtés sont déjà triés, souvent parce qu'un index les fournit dans l'ordre.

Théorie : `Theorie_10_SQL.md`, section 3.

9. Une condition est sargable quand le moteur peut la traduire en une descente d'index, c'est-à-dire quand elle compare la colonne elle-même à une valeur, sans lui appliquer de calcul.

Théorie : `Theorie_10_SQL.md`, section 4.

10. `WHERE orderdate >= '2023-01-01' AND orderdate < '2024-01-01'`. La réécriture a déplacé le calcul de la colonne vers la constante : l'index contient des dates, et il peut maintenant reconnaître ce qu'on cherche.

Théorie : `Theorie_10_SQL.md`, section 4.

11. Parce qu'un index B-tree est trié par le début de la chaîne. Chercher une fin de chaîne revient à chercher dans un annuaire tous les noms qui finissent par « eau », ce que l'ordre alphabétique ne permet pas.

Théorie : `Theorie_10_SQL.md`, section 4.

12. La cause est la collation de la base : l'index est trié selon les règles linguistiques de cette collation, et cet ordre ne correspond pas à celui que `LIKE` suppose. La parade est un index déclaré avec la classe d'opérateurs `text_pattern_ops`, qui trie octet par octet et sert donc les recherches par préfixe.

Théorie : `Theorie_10_SQL.md`, section 4.

13. Il a converti la colonne, donc lui a appliqué une fonction, donc a rendu l'index inutilisable. Il fallait convertir la constante et écrire `WHERE cle = '29825'`. Les deux écritures renvoient la même ligne, et seule la seconde descend l'index.

Théorie : `Theorie_10_SQL.md`, section 4.

14. Un index sur `(a, b, c)` se descend directement sur `a`, sur `a` et `b`, et sur les trois. Un filtre sur `b` seul ne peut pas y entrer d'un coup. C'est un annuaire trié par nom puis par prénom : on n'y trouve pas tous les Camille d'un seul regard sans connaître leur nom de famille.

PostgreSQL 18 s'en sert quand même, par le parcours par saut : il ouvre l'index à chaque groupe de valeurs de `a` et y cherche `b`, comme on ouvrirait l'annuaire à chaque nom de famille. Le plan compte ces descentes sous `Index Searches`, 247 au labo contre une seule pour un index dont `b` est la tête. Le saut dépanne une requête imprévue ; il ne remplace pas l'index bien ordonné.

Théorie : `Theorie_10_SQL.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

15. L'égalité en premier, la plage en dernier. Une plage disperse le tri : à l'intérieur d'un intervalle de dates, les valeurs de la colonne suivante ne sont plus rangées ensemble, donc cette colonne ne sert plus qu'à sauter d'une date à la suivante. Mesuré au labo sur le même filtre, 398 descentes pour l'index `(jour, customerid)`, une seule pour `(customerid, jour)`.

Théorie : `Theorie_10_SQL.md`, section 5.

16. Un index qui contient toutes les colonnes dont la requête a besoin, y compris celles du `SELECT`, si bien que le moteur n'ouvre jamais la table. `INCLUDE` permet d'ajouter des colonnes de réponse sans les faire entrer dans le tri : l'index grossit un peu, mais la recherche reste celle des colonnes de tête.

Théorie : `Theorie_10_SQL.md`, section 5.

17. Une table très écrite et peu lue ; une table petite, où le parcours séquentiel gagne ; une colonne peu sélective, qui ne réduit rien ; un index qui fait doublon avec le préfixe d'un autre. La parade du troisième cas est l'index partiel, restreint par un `WHERE` aux seules lignes qu'on cherche vraiment.

Théorie : `Theorie_10_SQL.md`, section 6.

18. On lit `pg_stat_user_indexes` et sa colonne `idx_scan`. Un compteur à zéro après plusieurs semaines d'exploitation désigne un index qu'on paie sans jamais le lire.

Théorie : `Theorie_10_SQL.md`, section 6.

19. Sur un résumé statistique des données, pas sur les données. Il se trompe après un chargement massif tant qu'`ANALYZE` n'a pas tourné, non qu'il croie la table vide, puisqu'il en tire la taille de son fichier, mais parce qu'il ignore la répartition des valeurs ; sur des colonnes corrélées dont il suppose l'indépendance ; et sur une distribution très irrégulière que l'échantillon par défaut décrit mal.

Théorie : `Theorie_10_SQL.md`, section 7.

20. Les métadonnées des blocs de lignes, qui donnent pour chaque bloc et chaque colonne la valeur minimale et la valeur maximale, permettent au moteur de sauter les blocs qui ne peuvent rien contenir d'intéressant. C'est l'élagage. Pour savoir s'il a pu jouer, on compte d'abord les blocs et on lit leurs bornes, par `PRAGMA_STORAGE_INFO` ; puis on compare le temps total d'`EXPLAIN ANALYZE`, relancé plusieurs fois. Le diagnostic est plus difficile parce que le plan ne change pas de forme : la requête élaguée et celle qui ne l'est pas ont le même plan au mot près, et seul le temps les sépare, là où PostgreSQL affiche `Seq Scan` contre `Index Scan`. Sur Snowflake, d'après sa documentation, le *Query Profile* affiche les partitions parcourues sur le total, et l'élagage s'y voit d'un coup d'œil.

Théorie : `Theorie_10_SQL.md`, section 8.

Questions pratiques

P1. `[OLTP]`

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Sans index.
EXPLAIN (ANALYZE, BUFFERS)
SELECT ligne_id, jour, linetotal
FROM   labo10.grosse
WHERE  customerid = 29825 AND jour >= '2023-01-01' AND jour < '2024-01-01'
ORDER  BY jour LIMIT 10;

CREATE INDEX idx_grosse_client_jour ON labo10.grosse (customerid, jour);
ANALYZE labo10.grosse;

-- Après.
EXPLAIN (ANALYZE, BUFFERS)
SELECT ligne_id, jour, linetotal
FROM   labo10.grosse
WHERE  customerid = 29825 AND jour >= '2023-01-01' AND jour < '2024-01-01'
ORDER  BY jour LIMIT 10;
```

Le rapport dépend de votre machine, et il se compte en dizaines. Ce qui compte davantage que le rapport, c'est le nombre de blocs lus, que `BUFFERS` donne : il passe de la table entière à quelques unités, et cette mesure-là ne dépend pas de l'état du cache.

Théorie : `Theorie_10_SQL.md`, sections 2 et 3.

P2. `[OLTP]`

```sql
-- Emploie l'index sur jour.
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse
WHERE  jour >= '2023-01-01' AND jour < '2024-01-01';

-- Ne l'emploie pas : fonction sur la colonne.
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse
WHERE  EXTRACT(YEAR FROM jour) = 2023;

-- Ne l'emploie pas non plus : conversion sur la colonne.
EXPLAIN (ANALYZE) SELECT SUM(linetotal) FROM labo10.grosse
WHERE  to_char(jour, 'YYYY') = '2023';
```

Les trois renvoient la même somme. Seule la première laisse l'index reconnaître ce qu'on cherche : son plan porte un `Index Cond`, les deux autres un parcours séquentiel et un `Filter`. Une réponse écrite avec `COUNT(*)` tombe dans le piège de la sous-partie 3.1 du labo : les trois plans y affichent un parcours d'index seul, et seule la ligne `Index Cond` distingue la descente de la lecture complète de l'index.

Théorie : `Theorie_10_SQL.md`, section 4. Théorie : `Theorie_02_SQL.md`, section 6.

P3. `[OLTP]`

© 2026 iAlexMG. Tous droits réservés.

```sql
CREATE INDEX idx_grosse_client_jour ON labo10.grosse (customerid, jour);
```

L'ordre est commandé par la nature des conditions, pas par la sélectivité. `customerid` est filtré par égalité, donc il va en tête : l'index se descend directement au bon client. `jour` est filtré par plage, donc il vient après, et à l'intérieur d'un client les jours sont rangés dans l'ordre, ce qui sert à la fois le filtre de plage et le `ORDER BY jour`, qui devient gratuit.

L'ordre inverse, `(jour, customerid)`, obligerait le moteur à sauter d'une date à la suivante dans toute la plage pour y chercher le client : mesuré au labo sur le même filtre, 398 descentes dans l'index au lieu d'une.

Théorie : `Theorie_10_SQL.md`, section 5.

P4. `[OLTP]`

```sql
CREATE INDEX idx_grosse_client_jour_couvrant
    ON labo10.grosse (customerid, jour) INCLUDE (ligne_id, linetotal);
ANALYZE labo10.grosse;
```

Le plan devient un `Index Only Scan` : toutes les colonnes demandées vivent dans l'index. En ajoutant `produit` au `SELECT`, la couverture est rompue et le moteur doit rouvrir la table pour chaque ligne retenue. Deux réponses sont alors défendables : ajouter `produit` au `INCLUDE`, ce qui grossit sensiblement l'index puisque c'est du texte, ou accepter le retour à la table. Une bonne réponse mentionne aussi le `Heap Fetches` : même couvert, l'index n'évite la table que sur les pages que le nettoyage a marquées, donc un `VACUUM` récent fait partie du résultat.

Théorie : `Theorie_10_SQL.md`, section 5.

P5. `[OLTP]`

```sql
SELECT relname                                          AS table_cible,
       indexrelname                                     AS index_,
       idx_scan                                         AS utilisations,
       pg_size_pretty(pg_relation_size(indexrelid))     AS taille
FROM   pg_stat_user_indexes
WHERE  schemaname = 'labo10'
ORDER  BY idx_scan, pg_relation_size(indexrelid) DESC;
```

Théorie : `Theorie_10_SQL.md`, section 6.

P6. `[OLTP]` (avancé)

```sql
CREATE INDEX idx_grosse_grosses_lignes
    ON labo10.grosse (customerid) WHERE linetotal > 1000;
ANALYZE labo10.grosse;

-- Servie : le prédicat de la requête implique celui de l'index.
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse
```


© 2026 iAlexMG. Tous droits réservés.

```sql
WHERE  customerid = 29825 AND linetotal > 1000;

-- Non servie : le prédicat est plus large que celui de l'index.
EXPLAIN (ANALYZE) SELECT COUNT(*) FROM labo10.grosse
WHERE  customerid = 29825 AND linetotal > 500;
```

La frontière est logique et non numérique : le planificateur emploie un index partiel seulement s'il peut prouver que la condition de la requête implique celle de l'index. `linetotal > 1000` implique `linetotal > 1000`, donc l'index sert ; `linetotal > 500` ne l'implique pas, puisqu'il désigne aussi des lignes absentes de l'index, donc l'index ne peut pas répondre seul.

Théorie : `Theorie_10_SQL.md`, sections 2 et 6.

P7. `[OLAP]`

```sql
-- Permet l'élagage.
SELECT COUNT(*) FROM dbo.FactInternetSales
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2013-02-01';

-- Ne le permet pas.
SELECT COUNT(*) FROM dbo.FactInternetSales
WHERE  YEAR(OrderDate) = 2013 AND MONTH(OrderDate) = 1;
```

On regarde le `Total Time` d'un `EXPLAIN ANALYZE`, relancé deux ou trois fois pour écarter le bruit, et rien d'autre : le plan des deux requêtes est identique, seule la durée diffère. Sur la table triée du labo, l'écart est net et stable : la version qui applique `YEAR` et `MONTH` met environ deux fois le temps de celle qui filtre par intervalle, pour le même compte. La raison est que les bornes conservées par le moteur portent sur la valeur de `OrderDate` et non sur le résultat d'un appel de fonction.

Sur `FactInternetSales` elle-même, il n'y a rien à élaguer : ses 60 398 lignes tiennent en un seul bloc, mesuré par `PRAGMA_STORAGE_INFO`, et l'écart qui reste ne vient que du calcul de `YEAR` et de `MONTH` sur chaque ligne. Une bonne réponse le dit et refait la mesure sur la table du laboratoire, qui compte vingt blocs.

Théorie : `Theorie_10_SQL.md`, section 8.

P8. `[OLTP]` (avancé)

```sql
ANALYZE ma_table;                -- 1
VACUUM ANALYZE ma_table;         -- 2, si le chargement a fait des mises à jour
SELECT pg_size_pretty(pg_total_relation_size('ma_table'));   -- 3
SELECT * FROM pg_stat_user_tables WHERE relname = 'ma_table'; -- 4
```

La première rafraîchit les statistiques, sans quoi tout plan mesuré ensuite serait choisi sans statistiques de colonne : le planificateur tirerait la taille de la table de son fichier, mais appliquerait à toute égalité la même part par défaut, le produit rare comme le produit courant, ce que le labo mesure à la sous-partie 6.1. La deuxième nettoie les versions mortes et met à jour la carte de visibilité, ce qui conditionne les parcours d'index seuls. La troisième donne l'ordre de grandeur, qui détermine ce qu'on peut espérer. La quatrième dit ce que la table a réellement subi, en insertions et en mises à jour, ce qui oriente le reste du diagnostic.

© 2026 iAlexMG. Tous droits réservés.

Est également acceptable une réponse qui commence par créer les index avant de charger plutôt qu'après, à condition de reconnaître le compromis : charger sur une table indexée est nettement plus lent, et l'ordre inverse est presque toujours préférable sur un chargement initial.

Théorie : `Theorie_10_SQL.md`, section 7.

Diagnostics

E1. Le planificateur estime 139 lignes sur le `Bitmap Heap Scan` et en trouve 42 440, soit un facteur de trois cents. Tout le reste découle de là.

Il faut lire l'estimation dans le bon sens. Cent trente-neuf lignes attendues, c'est peu : sur un si petit volume, une boucle imbriquée est le bon plan, puisqu'elle ne fera que 139 allers-retours dans l'index de la clé primaire des en-têtes. Le moteur a donc raisonné juste sur des chiffres faux. La sanction arrive à l'exécution, quand les allers-retours deviennent 42 440 : les lignes `loops=42440` et `Index Searches: 42440` le disent, et ce seul nœud touche 127 255 pages. Une sous-estimation ne fait pas éviter l'index, elle fait choisir les plans qui ne tiennent que sur de petits volumes, la boucle imbriquée en tête.

Le plan montre aussi où l'estimation se perd. Le `Bitmap Index Scan`, qui ne porte que sur `productid`, estime 48 122 lignes pour 46 880 réelles : l'index n'y est pour rien. C'est l'ajout du nom et du prix qui fait tomber l'estimation à 139. Le planificateur suppose les trois conditions indépendantes et multiplie leurs sélectivités, alors que le nom est entièrement déterminé par le produit, et le prix presque.

Ce n'est donc pas un problème d'index. On déclare la dépendance :

```
CREATE STATISTICS st_produit (dependencies) ON productid, produit, unitprice FROM labo10.grosse;
ANALYZE labo10.grosse;
```

Mesuré : l'estimation passe à environ 15 000 lignes, encore trois fois sous le réel, parce que le prix ne dépend pas tout à fait du produit (4 440 des 46 880 lignes du produit 870 sont vendues à un autre prix). C'est assez pour que le planificateur écarte la boucle imbriquée et revienne au `Hash Join`, qui lit une fois chaque table : moins de la moitié du temps, et 4 629 pages au lieu de 131 227. Une bonne réponse ajoute la remarque de fond : ce qu'il fallait corriger, c'est l'information, pas le plan. Le planificateur ne se trompe presque jamais sur ce qu'il faut faire d'un volume donné ; il se trompe sur le volume.

Théorie : `Theorie_10_SQL.md`, sections 3 et 7.

E2. `jour + INTERVAL '1 day'` applique un calcul à la colonne, donc l'index sur `jour` ne peut pas servir. On déplace le calcul du côté de la constante :

```
SELECT COUNT(*) FROM commande WHERE jour > now() - INTERVAL '1 day';
```

`now()`, l'instant présent, est le `CURRENT_TIMESTAMP` de PostgreSQL (Cours 02, section 6).

Théorie : `Theorie_10_SQL.md`, section 4.

E3. `Index Searches` compte les descentes dans l'arbre, depuis sa racine. Une recherche servie par la tête de l'index en fait une. Ici, l'index commence par `jour`, et ni A ni B ne filtre `jour` par égalité : aucune ne peut y entrer d'un coup par le client, c'est la règle du préfixe le plus à gauche. PostgreSQL 18 contourne

© 2026 iAlexMG. Tous droits réservés.

l'obstacle par le parcours par saut : il avance d'un jour au suivant et redescend chercher le client 29825 dans chacun. Pour A, sur toute l'étendue de l'index, 414 descentes ; pour B, sur les jours depuis 2023, 398 descentes et environ 1 300 pages lues.

L'auteur a donc raison sur la forme et tort sur le fond. L'index sert, au sens où le plan l'emploie et bat la lecture de la table, mais il sert comme un annuaire qu'on ouvrirait à chaque nom de famille. L'index à créer met l'égalité devant :

```
CREATE INDEX idx_client_jour ON labo10.grosse (customerid, jour);
```

Mesuré au labo, il sert A et B en une seule descente, et une poignée de pages d'index. Il rend le premier inutile pour ces deux requêtes.

Théorie : `Theorie_10_SQL.md`, section 5.

E4. Le parcours d'index prend 0,003 milliseconde par itération, et le plan indique `loops=905`. Son coût est donc d'au moins 2,7 millisecondes, et le nœud parent en dit davantage : la boucle finit à 7,6 millisecondes, dont 2,2 pour le parcours des en-têtes, ce qui laisse plus de 5 millisecondes à la boucle interne. Le temps n'est pas ailleurs, il est là : c'est le nœud le plus lourd du plan. L'affichage au millième explique l'écart entre les deux calculs, et c'est une raison de lire le parent plutôt que de multiplier un temps arrondi.

Le compte de lignes se lit de la même façon. `rows=22.62` est une moyenne par boucle, d'où les décimales de PostgreSQL 18 : 905 boucles de 22,62 lignes font les 20 470 lignes que le nœud parent rend.

La boucle imbriquée était pourtant le bon choix : la table externe est petite, 905 commandes, et la table interne est indexée sur la colonne de jointure. Mesuré en l'interdisant (`SET enable_nestloop = off`, comme à la sous-partie 6.3), le planificateur passe à une jointure de hachage qui lit toute la table `grosse` : environ dix fois le temps, et près de 16 000 pages au lieu de 4 558. Ce qui était faux, c'était la lecture du plan, pas le plan.

Théorie : `Theorie_10_SQL.md`, section 3.

E5. La démarche, dans l'ordre.

D'abord mesurer avant de toucher à quoi que ce soit : on relève le débit d'insertion actuel, pour avoir un point de comparaison qui survive aux modifications.

Ensuite chercher qui sert. La première requête à écrire est celle de `pg_stat_user_indexes` sur cette table, triée par `idx_scan` croissant. Sur onze index et un seul rapport quotidien, il est très probable que la majorité affiche un compteur ridicule.

Ensuite chercher les doublons : un index sur `(a)` en présence d'un index sur `(a, b)` est redondant, et se supprime sans rien perdre.

Enfin, examiner ce qui reste au regard du rapport quotidien. Un rapport qui tourne une fois par jour peut très bien se contenter d'un parcours séquentiel de quelques minutes, et la question à poser est celle du compromis global, pas celle de la vitesse du rapport.

Une bonne réponse ajoute la précaution : on supprime les index un par un, en gardant leur définition, et on surveille le rapport après chaque suppression.

Théorie : `Theorie_10_SQL.md`, section 6.

E6. Deux causes plus probables que le rangement.

© 2026 iAlexMG. Tous droits réservés.

La première est la sargabilité : si le filtre applique une fonction à la colonne de date, aucune métadonnée ne peut éliminer quoi que ce soit. On le vérifie en lisant la requête, et en la réécrivant en intervalle semi-ouvert.

La seconde est la taille de la table : sous quelques blocs de lignes, il n'y a rien à élaguer, et lire « tous » les blocs peut vouloir dire en lire un. `FactInternetSales`, avec ses 60 398 lignes, n'en compte qu'un, mesuré. On le vérifie en comptant les `row_group_id` distincts de `PRAGMA_STORAGE_INFO`, avant de regarder les temps.

Une troisième cause mérite d'être citée : le filtre peut porter sur une colonne différente de celle qui ordonne la table. Les bornes des blocs le montrent, quand chacun couvre toute la période. Réécrire la table dans un autre ordre est alors bien la bonne réponse, ou, sur une plateforme qui l'offre, une clé de clustering, comme sur Snowflake d'après sa documentation ; mais il faut savoir sur quelle colonne, et sur quelle fréquence d'interrogation on justifie la réécriture.

Théorie : `Theorie_10_SQL.md`, section 8.

Questions à développer

A1. Une bonne réponse tient l'image jusqu'au bout. Un achat se paie une fois et se garde ; un abonnement se prélève à chaque écriture, indéfiniment, et son intérêt dépend entièrement de l'usage qu'on en fait. Un index ne devient rentable que si la fréquence des lectures qu'il accélère dépasse le coût des écritures qu'il ralentit, et ce rapport change avec le temps : un index créé pour un rapport abandonné depuis deux ans continue d'être prélevé.

Les données à rassembler pour décider d'une suppression sont au nombre de quatre : le compteur `idx_scan` sur une période représentative, la taille de l'index, le volume d'écritures de la table, et la liste des autres index dont il pourrait être le préfixe redondant. La période représentative compte plus que le reste : un index qui ne sert qu'à la clôture trimestrielle affiche zéro pendant onze semaines.

Avant la suppression, on garde la définition exacte de l'index, ce que la colonne `indexdef` de `pg_indexes` donne, pour pouvoir le recréer à l'identique. PostgreSQL n'offre pas de commande pour rendre un index invisible au planificateur le temps d'un essai : `ALTER INDEX … INVISIBLE` rend une erreur de syntaxe, vérifié. Sur un autre moteur, cherchez dans sa documentation une telle commande avant de supprimer quoi que ce soit.

L'essai se fait autrement sur PostgreSQL, parce que son DDL s'annule. On retire l'index dans une transaction, on regarde les plans des requêtes qui comptent, et on annule :

```sql
BEGIN;
DROP INDEX labo10.idx_grosse_produit;
EXPLAIN SELECT COUNT(*) FROM labo10.grosse WHERE productid = 897;
ROLLBACK;
```

Mesuré : le plan passe au parcours séquentiel pendant la transaction, et l'index est de retour après le `ROLLBACK`, sans reconstruction. Le prix est le verrou : `pg_locks` montre que le `DROP INDEX` tient un verrou d'accès exclusif sur la table jusqu'à la fin de la transaction, le même que celui du `TRUNCATE` au Cours 11, qui bloque aussi les lectures. En production, on se contente donc d'`EXPLAIN` sans `ANALYZE`, et on annule vite.

Théorie : `Theorie_10_SQL.md`, section 6. Théorie : `Theorie_01_SQL.md`, section 2.

© 2026 iAlexMG. Tous droits réservés.

A2. Ce que la règle a de juste : elle vise le bon endroit. Les colonnes filtrées sont bien celles qui peuvent tirer profit d'un index, et une colonne jamais filtrée n'a jamais besoin d'être indexée. La règle capte aussi le fait qu'un index manquant est un problème silencieux, qui se paie sur toutes les requêtes à la fois.

Ce qu'elle rate est le reste. Elle ignore la sélectivité, donc elle indexerait une colonne à trois valeurs. Elle ignore les index composites, donc elle créerait quatre index simples là où un seul index à deux colonnes ferait mieux. Elle ignore le coût en écriture, donc elle ruinerait une table d'ingestion. Et elle ignore la fréquence, donc elle indexerait pour une requête lancée deux fois par an.

Une règle de remplacement acceptable : on indexe les colonnes filtrées par égalité dans les requêtes qui comptent, en préférant un index composite bien ordonné à plusieurs index simples ; puis on vérifie par la mesure, et on supprime ce qui ne sert pas.

Théorie : `Theorie_10_SQL.md`, section 6.

A3. PostgreSQL demande une structure explicite parce que ses données sont rangées par ligne et dans l'ordre où elles sont arrivées : rien dans la table ne dit où se trouve une valeur, et il faut donc construire à côté une structure triée qui le dit. Cette structure appartient à celui qui la crée : il en choisit les colonnes, l'ordre, l'étendue, et il en paie le loyer.

L'entrepôt n'en demande pas parce que ses données sont rangées par colonne, en blocs dont le moteur conserve les bornes. Ces bornes suffisent à écarter des blocs entiers, donc l'essentiel du travail d'un index se fait sans qu'on ait rien à déclarer.

Ce que le second ne dispense pas de faire, et c'est le point qui distingue une bonne réponse : les bornes ne servent que si les valeurs sont regroupées physiquement. Le labo le mesure sur deux tables au contenu identique : celle qui est mêlée met environ quatre fois le temps de celle qui est triée par date. Une table dont les dates sont dispersées dans tous les blocs ne s'élague pas, quoi qu'on écrive. Le levier existe, c'est de ranger la table sur la colonne qu'on filtre, et il se paie en réécriture plutôt qu'en espace. Les grandes plateformes automatisent ce rangement par une clé de clustering, qui se paie alors en calcul continu.

La seule notion qui vaut identiquement des deux côtés est la sargabilité. Une fonction appliquée à la colonne filtrée empêche le parcours d'index d'un côté et l'élagage de l'autre, pour exactement la même raison : la structure connaît la valeur de la colonne, pas le résultat d'un calcul.

Théorie : `Theorie_10_SQL.md`, section 8.

A4. L'écart entre l'estimation et le réel est plus informatif que le temps parce qu'il désigne une cause, alors que le temps ne fait que constater un symptôme. Un temps élevé peut venir de la taille des données, d'une machine chargée, d'un cache froid, ou d'un plan mauvais ; l'écart, lui, ne peut venir que d'une chose, à savoir que le planificateur a décidé sur une description fausse de la réalité. C'est aussi la seule information du plan qui reste valable quand on change de machine.

Trois causes possibles d'un facteur cent.

Les statistiques sont périmées, typiquement après un chargement massif ou une suppression massive. Le traitement est un `ANALYZE`, et la vraie correction est de l'ajouter au script d'alimentation.

Les colonnes filtrées sont corrélées, et le planificateur multiplie des sélectivités qu'il croit indépendantes. Le traitement est une déclaration de statistiques étendues sur le groupe de colonnes.

© 2026 iAlexMG. Tous droits réservés.

La distribution est très irrégulière, avec quelques valeurs qui écrasent les autres, et l'échantillon par défaut ne les décrit pas assez finement. Le traitement est d'augmenter la finesse de l'échantillon sur cette colonne, puis de relancer `ANALYZE`.

Une quatrième cause est également recevable : le prédicat est écrit d'une façon que le planificateur ne sait pas estimer, par exemple un appel de fonction dont il ignore la sélectivité. Le traitement est alors de réécrire le prédicat.

Théorie : `Theorie_10_SQL.md`, section 7.

Mises en situation

M1. Réponse défendable.

La démarche commence par le constat, pas par l'hypothèse. On récupère le plan réel du rapport, avec `EXPLAIN (ANALYZE, BUFFERS)`, et on le compare au comportement attendu. On regarde en priorité l'écart entre lignes estimées et lignes réelles, et le type de nœuds : un plan qui est passé d'une jointure de hachage à une boucle imbriquée est le symptôme classique d'une estimation qui a dérivé.

Trois hypothèses compatibles avec l'énoncé.

Les statistiques ont dérivé. La table a grossi de trente pour cent et une nouvelle application y écrit en continu ; si le démon de maintenance n'arrive plus à suivre, le planificateur travaille sur une description ancienne. La mesure qui tranche est la date du dernier `ANALYZE`, que `pg_stat_user_tables` donne, et le test est un `ANALYZE` manuel suivi d'une nouvelle mesure.

La table est gonflée par les versions mortes. La nouvelle application écrit en continu, donc elle produit des mises à jour, donc des versions périmées que le nettoyage doit récupérer. Une table gonflée se parcourt plus lentement à contenu égal. La mesure est la comparaison entre la taille physique et le nombre de lignes, et le test est un nettoyage.

Le basculement de plan par franchissement de seuil. À trente pour cent de lignes en plus, une condition qui gardait quinze pour cent de la table en garde peut-être trente, et le planificateur abandonne l'index pour un parcours séquentiel. C'est un changement légitime de sa part, et la correction est du côté de la requête ou de l'index, pas du côté du planificateur.

Ce qu'il ne faut pas faire en premier est d'ajouter un index, et c'est la partie qui distingue une bonne réponse. Aucun élément de l'énoncé ne désigne un index manquant, puisque le SQL n'a pas changé et que le rapport fonctionnait. Ajouter un index reviendrait à traiter un symptôme dont on n'a pas identifié la cause, et à ajouter un loyer permanent sur une table déjà très écrite.

Théorie : `Theorie_10_SQL.md`, sections 3 et 7.

M2. Réponse défendable.

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on demande avant d'agir : les requêtes réellement émises par la page, avec leur fréquence et leur plan. Une page à quatre filtres facultatifs n'émet pas une requête mais une famille de requêtes, et on ne peut rien décider sans savoir lesquelles dominent. On demande aussi ce que veut dire « lente », en chiffres, et sur quel percentile.

Pourquoi un index sur `date_livraison` seul a peu de chances de suffire : la plage de dates est rarement sélective à elle seule sur 90 millions de lignes, et si l'utilisateur remplit deux champs, un index à une colonne oblige le moteur à remonter beaucoup de lignes avant de filtrer le reste. Le coût est en plus payé sur 400 écritures par seconde.

La stratégie proposée tient en deux idées. D'abord, un petit nombre d'index composites, construits sur les combinaisons réellement fréquentes plutôt que sur les colonnes prises une à une, avec l'égalité en tête et la plage de dates en queue : un index sur `(entrepot, statut, date_livraison)` sert à lui seul les filtres sur l'entrepôt, sur l'entrepôt et le statut, et sur les trois. Ensuite, on refuse d'indexer ce qui ne sera pas filtré assez souvent pour valoir son loyer.

Le nom de client partiel est le cas délicat, et il mérite un traitement séparé. S'il s'agit d'une recherche par préfixe, un index avec la classe d'opérateurs de motif suffit. S'il s'agit d'une recherche par fragment, aucun B-tree ne servira, et il faut un index spécialisé pour la recherche textuelle. Une troisième réponse est souvent la meilleure : exiger de l'interface qu'un autre filtre soit rempli avant de lancer une recherche par fragment sur 90 millions de lignes.

Après la mise en place, on surveille trois choses : le temps de la page au percentile choisi, le débit d'écriture, et le compteur d'utilisation de chaque index créé. Le troisième est celui qu'on oublie, et c'est celui qui dira dans six mois lesquels supprimer.

Théorie : `Theorie_10_SQL.md`, sections 5 et 6.

M3. Réponse défendable.

Il n'y a aucune liste d'index à recréer, parce qu'un entrepôt n'en a pas d'utile et qu'il n'y a rien à remplacer un pour un. Ce qui remplace l'index, ce sont les métadonnées de bloc, qui donnent au moteur les bornes de chaque colonne dans chaque bloc et lui permettent d'écarter ce qui ne peut rien contenir d'utile. Ce mécanisme est automatique et gratuit, et il n'y a rien à déclarer.

Ce qui se transpose est plus important que ce qui ne se transpose pas, et c'est le vrai contenu de la réponse. L'équipe a passé trois ans à apprendre quelles colonnes sont filtrées, dans quel ordre, combien de lignes chaque prédicat garde, et quelles requêtes comptent. Cette connaissance est le travail difficile, et elle reste intégralement valable : c'est elle qui dira sur quelle colonne ranger chaque grosse table, quelles requêtes réécrire, et où placer les efforts. Ce qui devient sans objet est la partie mécanique, à savoir la fabrication des index, leur ordre de colonnes, leur maintenance et leur surveillance.

Les décisions à prendre réellement sur l'entrepôt sont peu nombreuses. Le rangement des deux ou trois plus grosses tables, décidé sur la colonne la plus souvent filtrée par plage : une réécriture triée sur un entrepôt comme celui de la formation, ou une clé de clustering sur une plateforme qui en offre, comme Snowflake d'après sa documentation. Sur une plateforme infonuagique, la puissance de calcul qu'on y loue, l'entrepôt de calcul de Snowflake par exemple, levier direct et souvent mal compris. L'organisation des données en couches, pour que les rapports lisent des tables préparées plutôt que la matière brute. Et l'écriture des requêtes elles-mêmes, où la sargabilité continue de compter exactement comme avant.

Chacune se valide par la mesure. On prend les dix requêtes qui coûtent le plus, on note pour chacune le temps passé par étape et le rangement des tables qu'elle lit, que DuckDB donne par `EXPLAIN ANALYZE` et `PRAGMA_STORAGE_INFO`, et Snowflake, d'après sa documentation, avec les partitions parcourues sur le total dans son *Query Profile* ; on applique une modification à la fois, et on recompare. C'est la même discipline que sur PostgreSQL, avec un autre instrument.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_10_SQL.md`, section 8.

© 2026 iAlexMG. Tous droits réservés.